

CITREA · SECURITY REVIEW

Citrea
Token.

A security review of Token.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Citrea	Token	Guardian	February 27 – March 9, 2026	March 16, 2026

§ 00

Contents.

25 pages — overview, confidence ranking, scope, 12 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 9 findings	P. 08
	Remediation Review · 3 findings	P. 19
§ 05	Disclaimer	P. 24
§ 06	About Guardian	P. 25

§ 01

Overview.

Guardian reviewed Token across 2 review rounds. The final report includes 2 Low, 10 Informational findings.

AUDIT FIRM	Guardian	REPOSITORY	chainwayxyz/citrea-token
CLIENT	Citrea	METHODOLOGY	Smart contract · Manual · Main Review · Remediation Review
PROJECT	Token	MAIN REVIEW COMMIT	0230630de9d4
PREPARED BY	Guardian	REMEDATION REVIEW COMMIT	00060a0b9b4f
AUDIT WINDOW	February 27 – March 9, 2026		
FINAL REPORT	March 16, 2026		

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	0	0	0	0	0
Low	2	0	0	0	2
Informational	10	0	5	0	5
All severities	12	0	5	0	7

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Token in `chainwayxyz/citrea-token`. The scope includes 396 source lines across 6 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/CitreaToken.sol	21	26
src/GaugeVotes.sol	35	55
src/xCitreaToken.sol	260	446
script/01_DeployCTR.s.sol	25	37
script/02_DeployXCTR.s.sol	48	63
config/xCTRConfig.sol	7	10

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
L-01	maxWithdraw/maxRedeem Returns Non-Zero Values	Unexpected Behavior	LOW	RESOLVED	p. 8
L-02	Orphaned Rewards Harm First Depositor	Rewards	LOW	RESOLVED	p. 9
I-01	Missing View Preview For Custom Withdrawals	Best Practices	INFORMATIONAL	RESOLVED	p. 11
I-02	RewardsAdded Emits Exiter As Sender	Events	INFORMATIONAL	RESOLVED	p. 13
I-03	deposit(0) Succeeds Silently	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 14
I-04	Exit Penalty Rounds Down Favoring Exiter	Rounding	INFORMATIONAL	RESOLVED	p. 15
I-05	cancelExit Mints Zero Shares On Inflation	Validation	INFORMATIONAL	RESOLVED	p. 16
I-06	ERC4626 First Depositor Inflation Attack	Math	INFORMATIONAL	ACKNOWLEDGED	p. 17
I-07	No Pause Mechanism For Emergency Response	Best Practices	INFORMATIONAL	RESOLVED	p. 18
REMEDIATION REVIEW					
I-01	Zero-Supply Breaks Cancels And Deposits	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 19
I-02	Misleading ERC4626 Deposit And Mint Limits	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 21
I-03	Pausing Can Lock Users Mid-Exit	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 23

LOW **L-01**

RESOLVED

maxWithdraw/maxRedeem Returns Non-Zero Values

CATEGORY	Unexpected Behavior
LOCATION	src/xCitreaToken.sol: 316-317 - maxWithdraw(), maxRedeem()
REVIEW	Main Review

DESCRIPTION

The `withdraw()` and `redeem()` functions are overridden to always revert with 'xCTR: Use startExit'. However, `maxWithdraw()` returns `convertToAssets(maxRedeem(owner))` and `maxRedeem()` returns `balanceOf(owner)` — both non-zero for users with xCTR shares.

Per ERC4626 specification:

- `maxWithdraw` MUST return 0 if withdraw would revert for the given owner
- `maxRedeem` MUST return 0 if redeem would revert for the given owner

This breaks ERC4626 composability. Integrators (aggregators, routers, yield optimizers) querying these functions would believe withdrawal is possible, construct transactions accordingly, and have them revert.

RECOMMENDATION

Override both to return 0:

```
function maxWithdraw(address) public pure override returns (uint256) {  
    return 0;  
}  
function maxRedeem(address) public pure override returns (uint256) {  
    return 0;  
}
```

RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

Orphaned Rewards Harm First Depositor

CATEGORY	Rewards
LOCATION	src/xCitreaToken.sol – addRewards() and totalAssets()
REVIEW	Main Review

DESCRIPTION

The `addRewards()` function does not check whether `totalSupply() > 0`. If rewards are added when there are no stakers, the rewards inflate `totalAssets()` without any shares to distribute them to.

When the reward epoch elapses and a user deposits:

1. `totalAssets()` = orphanedRewards (large value from streamed rewards)
2. `totalSupply()` = 0 (no stakers)
3. User's shares = $\text{deposit} * (0 + 1) / (\text{orphanedRewards} + 1) \rightarrow$ rounds to 0
4. User's CTR is transferred in but they receive 0 shares — permanent loss

No attacker required. Realistic scenarios:

- Governance adds rewards before the first staker joins
- All stakers exit while an automated reward distribution contract continues
- A delayed reward transaction lands after the last staker exits

PoC: Add 100 CTR rewards to empty vault, wait 14 days for full streaming, victim deposits 100 CTR → receives 0 shares, loses everything.

RECOMMENDATION

Add a supply check to `addRewards`:

```
function addRewards(uint256 amount) external {
    require(totalSupply() > 0, 'xCTR: No stakers');
    _addRewards(amount);
    SafeERC20.safeTransferFrom(IERC20(asset()), msg.sender, address(this), amount);
}
```

Alternatively, mint dead shares in `initialize()` to prevent the zero-supply state from ever occurring.

L-02 Orphaned Rewards Harm First Depositor

CONTINUED

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-01

RESOLVED

Missing View Preview For Custom Withdrawals

CATEGORY Best Practices

LOCATION src/xCitreaToken.sol

REVIEW Main Review

DESCRIPTION

The vault replaces standard ERC4626 withdrawal mechanics with a custom delayed-exit lifecycle through `startExit`, `completeExit` and `cancelExit`, but it does not provide a dedicated view function that returns expected net assets for a delayed withdrawal path at a chosen completion time. Integrators can estimate the instant path using `convertToAssets` and `getInstantExitPenalty`, but the delayed path requires either executing a state-changing `startExit` first or re-implementing internal penalty math and per-exit snapshot logic off-chain.

This creates integration friction and increases the chance of quote mismatch between user interfaces, bots and on-chain behavior. Because delayed exits depend on snapshotted per-exit parameters and elapsed time, the absence of a canonical view method means every integrator must duplicate protocol logic to forecast outcomes. In practice, duplicated logic tends to drift over time, especially after governance parameter changes or future upgrades, producing inaccurate UX and failed expectations.

RECOMMENDATION

Add explicit read-only quote functions for the custom exit model. A practical approach is to expose one view that previews delayed exit payout for a hypothetical new exit and one view that previews payout for an existing pending exit at an arbitrary timestamp.

```
function previewStartExit(uint256 shares)
external
view
returns (uint256 assetsLocked, uint256 penaltyAtMinDuration, uint256 netAtMinDuration);
function previewCompleteExit(address user, uint256 exitId, uint256 timestamp)
external
view
returns (uint256 assetsLocked, uint256 penalty, uint256 netAssets);
```

These helpers should use the same internal math and snapshotted fields as execution paths so integrators can rely on canonical on-chain previews without state mutation.

I-01 Missing View Preview For Custom Withdrawals

CONTINUED

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

RESOLVED

RewardsAdded Emits Exiter As Sender

CATEGORY	Events
LOCATION	src/xCitreationToken.sol: 341 - _addRewards()
REVIEW	Main Review

DESCRIPTION

The `_addRewards()` function emits `RewardsAdded(msg.sender, amount)` regardless of call context. When called from `addRewards()` (L137), `msg.sender` is the actual external reward contributor who transfers tokens in — correct. However, `_addRewards()` is also called from `completeExit()` (L175) and `instantExit()` (L211) for penalty redistribution. In these exit paths, `msg.sender` is the exiting user, not a reward contributor. No new tokens enter the contract — the penalty is simply reallocated from the exiting user's locked assets to the reward pool. Yet the event is identical to a genuine external reward addition.

Off-chain indexers, dashboards, and analytics tools that track `RewardsAdded` events will incorrectly attribute penalty reallocations as external reward contributions from exiting users. This leads to inflated reward contribution metrics and incorrect reward source accounting (internal penalty vs external deposit). No funds at risk — only emitted event metadata is misleading.

RECOMMENDATION

Emit a distinct event for penalty-sourced rewards:

```
event PenaltyRedistributed(address indexed exiter, uint256 amount);  
function _addRewards(uint256 amount) internal {  
  _addRewardsInternal(amount);  
  emit RewardsAdded(msg.sender, amount);  
}  
function _addPenaltyRewards(uint256 amount) internal {  
  _addRewardsInternal(amount);  
  emit PenaltyRedistributed(msg.sender, amount);  
}
```

RESOLUTION

Status: Resolved.

INFORMATIONAL I-03

ACKNOWLEDGED

deposit(0) Succeeds Silently

CATEGORY	Validation
LOCATION	src/xCitreaToken.sol – inherited deposit()
REVIEW	Main Review

DESCRIPTION

Calling `deposit(0, receiver)` succeeds without reverting. It mints 0 shares and emits a misleading `Deposit(sender, receiver, 0, 0)` event. The inherited OZ ERC4626 implementation does not guard against zero-amount deposits.

RECOMMENDATION

Override deposit to reject zero amounts:

```
function deposit(uint256 assets, address receiver) public override returns (uint256) {  
    require(assets > 0, 'xCTR: Zero deposit');  
    return super.deposit(assets, receiver);  
}
```

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

RESOLVED

Exit Penalty Rounds Down Favoring Exiter

CATEGORY	Rounding
LOCATION	src/xCitreaToken.sol: 266 – getExitPenalty()
REVIEW	Main Review

DESCRIPTION

The exit penalty calculation in `getExitPenalty()` uses `Math.mulDiv` with default floor rounding:

```
return Math.mulDiv(assets, scaledPenalty, BPS * durationRange);
```

This rounds the penalty DOWN, meaning the exiting user pays slightly less than they should. In DeFi protocols, fees and penalties should round UP (in favor of the protocol/remaining stakers) to prevent value leakage.

Impact is 1 wei per exit. Cumulative impact is negligible even with millions of exits.

RECOMMENDATION

Use ceiling rounding for the penalty calculation:

```
return Math.mulDiv(assets, scaledPenalty, BPS * durationRange, Math.Rounding.Ceil);
```

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-05**

RESOLVED

cancelExit Mints Zero Shares On Inflation

CATEGORY	Validation
LOCATION	src/xCitreaToken.sol: 199 – cancelExit()
REVIEW	Main Review

— DESCRIPTION

When a user cancels a pending exit via `cancelExit()`, the function converts locked assets back to shares at the current exchange rate without checking that the resulting shares are non-zero:

```
uint256 shares = convertToShares(pendingExit.assets);  
// No require(shares > 0) check  
_mint(msg.sender, shares);
```

If the share price has been inflated (via direct CTR donation or reward injection to empty vault), `convertToShares` rounds down to 0. The user's pending exit assets are returned to the pool (increasing `totalAssets` for other stakers) but the user receives 0 shares back — effectively losing their entire position.

This is a secondary impact of the first-depositor inflation vector. It requires the share price to already be massively inflated, which itself requires an economically irrational griefing attack.

— RECOMMENDATION

Add a minimum shares check in `cancelExit`:

```
uint256 shares = convertToShares(pendingExit.assets);  
require(shares > 0, 'xCTR: Zero shares');  
_mint(msg.sender, shares);
```

Alternatively, store the original shares burned in `startExit` within the `PendingExit` struct and restore that exact amount on cancel, avoiding exchange rate dependency entirely.

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-06

ACKNOWLEDGED

ERC4626 First Depositor Inflation Attack

CATEGORY	Math
LOCATION	src/xCitreaToken.sol – deposit() (inherited from ERC4626Upgradeable), totalAssets()
REVIEW	Main Review

— DESCRIPTION

The xCitreaToken contract does not override `_decimalsOffset()`, which defaults to 0 in OpenZeppelin's ERC4626 implementation. This means the contract relies solely on the +1 virtual share/asset protection, which while making the attack non-profitable for the attacker (OZ's documented mitigation), still allows grieving of early depositors.

An attacker can: (1) Deposit 1 wei CTR to get 1 share, (2) Transfer large CTR directly to xCTR to inflate totalAssets, (3) Subsequent depositors receive 0 shares due to inflated exchange rate.

PoC shows: attacker deposits 1 wei + donates 100 CTR. Victim deposits 50 CTR and gets 0 shares (100% loss). However, attacker spends 100 CTR and can only redeem ~75 CTR, losing ~25 CTR net. With the 50% instant exit penalty, the attacker's total loss is ~62.5 CTR to grief a 50 CTR deposit.

Additionally, xCTR shares are non-transferable, preventing the attacker from selling inflated shares. OZ documents this as an accepted trade-off with `offset=0`: 'the default offset (0) makes it non-profitable even if an attacker is able to capture value from multiple user deposits.'

— RECOMMENDATION

1. Override `_decimalsOffset()` to return 3-6, making inflation attacks orders of magnitude more expensive than profitable.
2. As defense-in-depth, add `require(shares > 0)` in `deposit` to revert instead of silently minting 0 shares (note: this converts the attack from silent loss to DoS on small deposits, so it's secondary to the offset fix).
3. Alternatively, mint dead shares in `initialize()` to establish a minimum share price floor.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

RESOLVED

No Pause Mechanism For Emergency Response

CATEGORY	Best Practices
LOCATION	src/xCitreaToken.sol – contract-wide
REVIEW	Main Review

— DESCRIPTION

The xCitreaToken vault does not implement any pause functionality (e.g., OpenZeppelin PausableUpgradeable). If a critical vulnerability is discovered post-deployment, there is no way to temporarily halt deposits, exits, or reward additions while a fix is prepared and deployed via the upgrade mechanism.

The only emergency response available is upgrading the proxy implementation, which requires deploying new code, going through the proxy admin flow, and potentially waiting for any timelock delays. During this window, the vulnerable functions remain fully operational.

Given that the contract is upgradeable (TransparentUpgradeableProxy), adding pausability would complement the upgrade pattern by providing an immediate circuit breaker while a proper fix is developed and audited.

— RECOMMENDATION

Inherit PausableUpgradeable and add whenNotPaused modifiers to key state-changing functions:

- deposit / mint
- addRewards
- startExit
- instantExit

Ensure completeExit and cancelExit are NOT paused — users should always be able to withdraw assets already in the exit pipeline. This provides an instant circuit breaker that buys time for a proper upgrade fix.

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-01

ACKNOWLEDGED

Zero-Supply Breaks Cancels And Deposits

CATEGORY Unexpected Behavior
LOCATION xCitreaToken.sol
REVIEW Remediation Review

DESCRIPTION

`xCitreaToken` allows the last staker to burn the final real shares through `startExit()`, while the vault can still retain economic value in several forms. Residual assets can remain because ERC4626 conversions use virtual assets and virtual shares. Unreleased rewards can keep streaming into `totalAssets()` after supply has already reached zero. Direct CTR transfers to the vault also increase its raw backing. Once that happens, the contract enters a broken state where it still has value, but there are no live shares anchoring the exchange rate.

That state breaks `cancelExit()` as when a user starts an exit, the contract stores only the asset amount and later reconstructs the canceled position with `convertToShares(pendingExit.assets)`. If `totalSupply()` has already fallen to zero while `totalAssets()` is nonzero, that conversion can round down to zero even though the user is still inside the allowed cancellation window. The function then reverts with `xCTR: No shares to restore`, leaving the pending exit impossible to cancel.

The deployed configuration makes this path realistic rather than theoretical. `epochPeriod` is 7 days while `minExitDuration` is 15 days. A sole staker can start an exit for the full position while the vault has queued rewards scheduled for the next epoch. After 7 days those rewards fully become claimable assets, but the user still has 8 more days during which `cancelExit()` is supposed to remain available. In that interval the vault can have `totalSupply() == 0` and `totalAssets() > 0`, so `convertToShares(pendingExit.assets)` returns zero and the cancel path is bricked before the cancel deadline expires.

The same zero-supply state also creates a direct user-loss risk for future deposits. OpenZeppelin ERC4626 `deposit()` does not require the previewed share amount to be nonzero. When supply is zero and the vault already contains residual assets, `previewDeposit()` computes $\text{assets} * (0 + 1) / (\text{totalAssets} + 1)$. Any deposit that does not exceed the existing residual balance can therefore mint zero shares and become a pure donation to the vault. This means the zero-supply state is not only an accounting edge case. It becomes a liveness problem for exiting users and a loss-of-funds trap for later depositors.

I-01 Zero-Supply Breaks Cancels And Deposits

CONTINUED

The impact is that a fully exited vault cannot reliably restart or unwind. Users can lose the ability to cancel a pending exit during the stated cancellation window, and new depositors can transfer CTR into the vault without receiving any xCTR in return.

— RECOMMENDATION

The vault should never be allowed to reach `totalSupply() == 0` while any economically meaningful underlying value remains. A robust mitigation is to mint permanent dead shares during initialization so share supply can never fully disappear. Another acceptable approach is to add explicit zero-supply unwind logic that sweeps or neutralizes residual value when the last live shares are burned.

`cancelExit()` should also stop depending on a fresh ERC4626 conversion. The contract already knows how many shares were burned when the exit began, so that quantity should be stored in `PendingExit` and restored directly on cancel:

```
struct PendingExit {  
    uint256 startTime;  
    uint256 shares;  
    uint256 assets;  
    // ...  
}
```

In addition, `deposit()` and `mint()` should revert whenever the previewed share amount is zero. The required invariant is that no user action can leave the vault with zero supply and positive backing while any later user can still interact with ERC4626 entry or cancel paths.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Misleading ERC4626 Deposit And Mint Limits

CATEGORY	Unexpected Behavior
LOCATION	xCitreaToken.sol
REVIEW	Remediation Review

DESCRIPTION

`xCitreaToken` overrides `maxDeposit()` and `maxMint()` to return the absolute `ERC20VotesUpgradeable._maxSupply()`. That value is only the global cap used by the votes extension. It is not the amount of additional shares that can still be minted from the current vault state, and it does not reflect whether the vault is paused. As a result, the two ERC4626 limit functions no longer satisfy the normal integrator expectation that they represent a safe upper bound for a successful call.

This mismatch matters because ERC4626 routers and frontends routinely query `maxDeposit()` or `maxMint()` before deciding whether to submit a transaction. In this implementation, those functions can return a large nonzero value even when `totalSupply()` is already near the votes cap or while `deposit()` and `mint()` are disabled by `whenNotPaused`. A caller can therefore observe an apparently valid limit, submit a transaction within that limit, and still revert later when `_mint()` hits the votes cap or when the pause check rejects the operation.

This issue breaks composability with tooling that relies on ERC4626 limit introspection and can cause unnecessary reverts.

RECOMMENDATION

`maxMint()` should return only the remaining votes headroom rather than the absolute cap, and `maxDeposit()` should be derived from that remaining headroom under the current conversion rate. Both functions should also return zero while the vault is paused so that off-chain callers do not get an obviously misleading admission signal.

A minimal correction is:

```
function maxMint(address) public view override returns (uint256) {  
    if (paused()) return 0;  
    return ERC20VotesUpgradeable._maxSupply() - totalSupply();  
}
```

I-02 Misleading ERC4626 Deposit And Mint Limits

CONTINUED

`maxDeposit()` can then convert that remaining share headroom into assets using the live ERC4626 exchange rate. The required invariant is that a call within the reported `maxDeposit()` or `maxMint()` bound should not fail for reasons that are already knowable from current contract state.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Pausing Can Lock Users Mid-Exit

CATEGORY	Unexpected Behavior
LOCATION	src/xCitreaToken.sol:210,240
REVIEW	Remediation Review

DESCRIPTION

The remediation introduces `PausableUpgradeable` and applies `whenNotPaused` broadly. However, `completeExit()` and `cancelExit()` are also guarded by `whenNotPaused` (see `src/xCitreaToken.sol:210,240`). This means the owner can call `pause()` and block users from finalizing already-started exits or cancelling exits within the intended cancellation window.

Pausability is typically intended to block new inbound actions (`deposit()` / `mint()` / `startExit()` / `addRewards()`) while still allowing users to complete withdrawals/egress. As implemented, pausing can temporarily lock user funds mid-exit and makes users dependent on the owner calling `unpause()` before they can retrieve assets.

RECOMMENDATION

Keep pausability, but scope it to block new inbound actions while allowing fund egress.

Suggested approach:

- Keep `whenNotPaused` on: `deposit()`, `mint()`, `addRewards()`, `startExit()` (and optionally `instantExit()`)
- Remove `whenNotPaused` from: `completeExit()` and `cancelExit()` so users can always finalize an exit/cancel even during an incident.

If the protocol explicitly intends to pause exits too, document this governance/ops trust assumption and consider a separate, more strictly governed `withdrawalsPaused` flag (timelock/multisig) rather than reusing `pause()`.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.